Day 1: Introduction to Full Stack Development

Concepts, Architecture and Java Full Stack Technologies

1. What is Full Stack Development?

Full stack development is building both the **user-facing side** (frontend) and the **server-side** (backend, database) of a web application. A **full stack developer** can design the interface, write the business logic, store and retrieve data, and connect all of these layers into one working product.

The word "stack" refers to the set of technologies layered together: what the user sees, the logic that powers it, and the data store beneath it.

2. Frontend vs Backend vs Database

Layer	Purpose	Runs on	Examples
Frontend	What users see and interact with: pages, forms, buttons, layout	User's browser	HTML, CSS, JavaScript, React.js
Backend	Business logic, validation, security, APIs; talks to the database	Server	Java, Spring Boot
Database	Stores and retrieves data permanently	Database server	MySQL, PostgreSQL (SQL)

3. Client-Server Architecture

Web applications follow the **client-server model**. The **client** (usually a browser) sends a request; the **server** processes it and sends back a response. The two communicate using the **HTTP/HTTPS** protocol, and data is commonly exchanged as **JSON**.

- **Client:** requests resources and displays results.
- **Server:** listens for requests, applies logic, talks to the database, returns responses.
- **Stateless communication:** each request carries all the information the server needs.

Common HTTP methods in REST APIs

Method	Purpose	Example
GET	Read data	GET /api/users
POST	Create data	POST /api/users
PUT	Update data	PUT /api/users/5
DELETE	Remove data	DELETE /api/users/5

4. How a Web Application Works

Example: a user logs in to a website.

1. The user enters an email and password in a React login form and clicks **Login**.

2. JavaScript sends an HTTP **POST** request with the credentials (as JSON) to the backend.
3. A Spring Boot **Controller** receives the request and passes it to a **Service**.
4. The Service validates the data and asks the **Repository** to look up the user.
5. The Repository runs a **SQL** query against the database.
6. The database returns the matching record to the backend.
7. The backend checks the password and builds a JSON response (success or error).
8. React receives the response and updates the screen, for example showing the dashboard.

5. Architecture Diagram

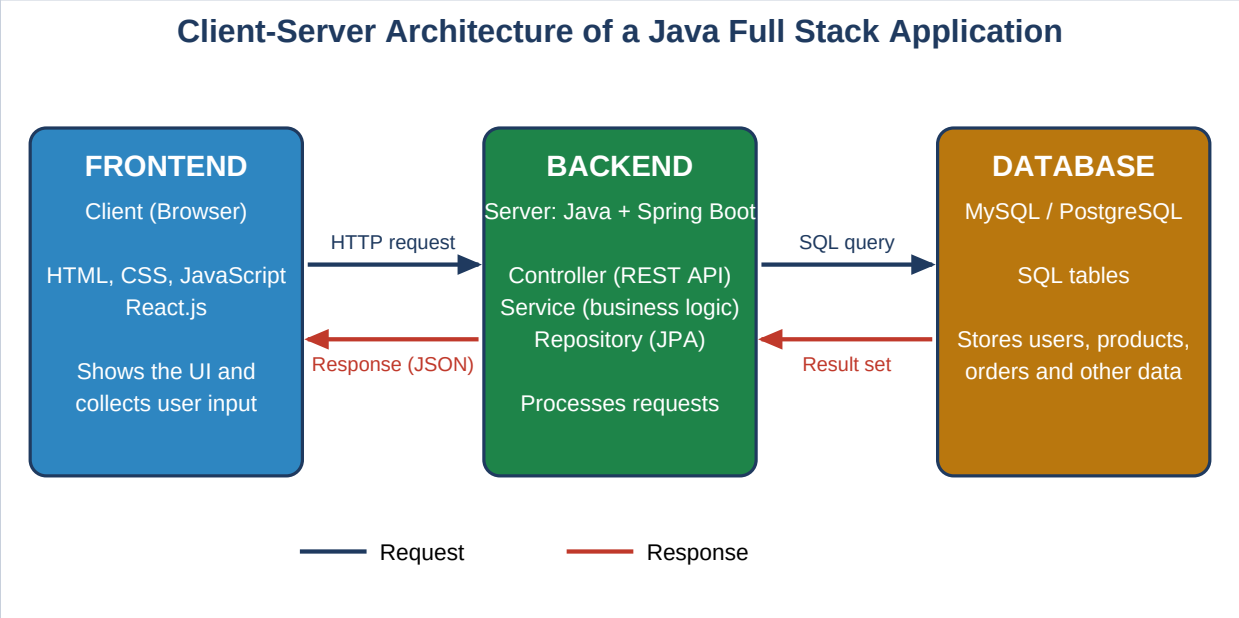

Figure 1: Frontend, backend and database communication

6. Introduction to Java Full Stack Development

Java full stack development uses **Java** and **Spring Boot** on the backend, **React.js** (with HTML, CSS and JavaScript) on the frontend, and a **SQL** database for storage.

Technology	Layer	Role
HTML	Frontend	Structure and content of web pages (headings, forms, links)
CSS	Frontend	Styling: colours, layout, fonts, responsive design
JavaScript	Frontend	Makes pages interactive; calls backend APIs
React.js	Frontend	Library for building component-based user interfaces
Java	Backend	General-purpose, object-oriented language for server logic
Spring Boot	Backend	Framework for building REST APIs quickly with minimal configuration
SQL	Database	Language to create, query and update relational data (MySQL, PostgreSQL)

7. Technologies in a Java Full Stack Application

Category	Technologies
Frontend	HTML5, CSS3, JavaScript (ES6), React.js, Axios or Fetch API
Backend	Java 17/21, Spring Boot, Spring Web (REST), Spring Data JPA, Hibernate, Spring Security
Database	MySQL or PostgreSQL, SQL, JDBC
Build tools	Maven or Gradle, npm
Version control	Git and GitHub
Tools	IntelliJ IDEA / VS Code, Postman, Node.js, embedded Tomcat server

Summary

- Full stack = frontend + backend + database.
- Browsers (clients) send HTTP requests; servers respond, usually with JSON.
- Java full stack: React.js for the UI, Spring Boot for the API, SQL for data.

Day 1 Checklist

[ ] Can explain frontend, backend and database in own words

[ ] Can describe the request-response cycle

[ ] Can draw the architecture diagram from memory

[ ] Can name the technology used at each layer